Lecture 13: Consensus with Paxos

CS 739: Distributed Systems

Prof. Andrea C. Arpaci-Dusseau
University of Wisconsin — Madison

Spring'26 Announcements

Reading Groups

- Pick your own Group 4 for rest of semester

Project 2 – Congrats!

3/11 + 3/3: Two In-class Midterm Exams

- Congrats!

Project 3 – Due 1 week AFTER Spring Break

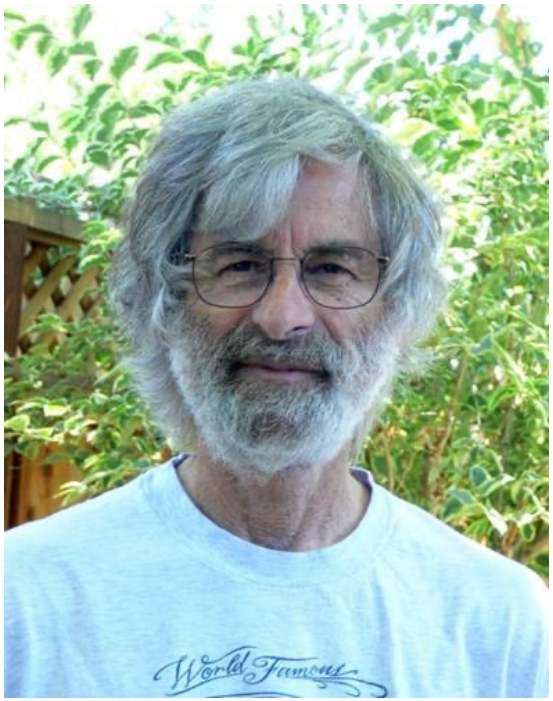
- Add consensus to KV-Store
- Form

Consensus: Paxos

Leslie Lamport. Paxos Made Simple. ACM SIGACT News, 2001.

Received 2013 Turing Award for "fundamental contributions to the theory and practice of distributed and concurrent systems, notably the invention of concepts such as causality and logical clocks, safety and liveness, replicated state machines, and sequential consistency”

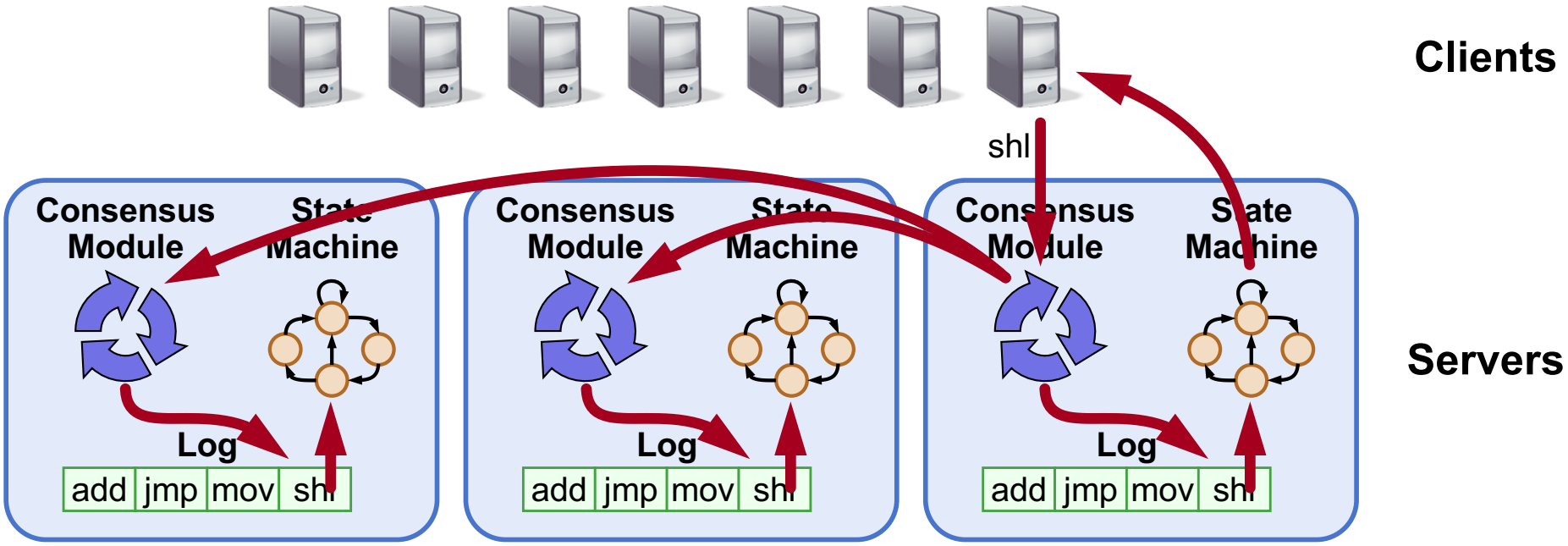
- The Byzantine Generals' Problem
- Time, Clocks, Ordering
- Paxos algorithm for consensus
- LaTex
- Temporal Logic (TLA and TLA+)



Implementing Replicated Logs with Paxos

Borrows heavily from slides by Lorenzo Alvisi, Ali Ghodsi, and David Mazières
John Ousterhout and Diego Ongaro

Goal: Replicated Log



- **Replicated log => replicated state machine**
  - All servers execute same commands in same order
- **Consensus module ensures proper log replication**
- **System makes progress as long as any majority of servers are up**
  - Given F failures, need 2F+1 servers (not just F+1 servers)

The Paxos Approach

Decompose the problem:

- **Basic Paxos ("single decree"):**
  - Simplest form of consensus
  - Agree on a single value
    - One or more servers propose values
    - System must agree on a single value as chosen
    - Only one value is ever chosen
- **Multi-Paxos:**
  - Agree on a log
  - Combine several instances of Basic Paxos to agree on a series of values forming the log
  - No clear specification
  - We will read Raft

Requirements for Basic Paxos

- **Safety:**
  - Only a single value may be chosen
  - A server never learns that a value has been chosen unless it really has been
- **Liveness (as long as majority of servers up and communicating with reasonable timeliness):**
  - Availability
  - Some proposed value is eventually chosen
  - If a value is chosen, servers eventually learn about it

Failure model: fail-stop / fail-recover crash and restart

- Messages reordered, dropped, and duplicated, network partitions
- What can't Paxos tolerate?

Paxos Components

- **Proposers:**
  - Active: put forth particular values to be chosen
- **Acceptors:**
  - Passive: respond to messages from proposers
  - Responses represent votes that form consensus
  - Store chosen value, state of the decision process
  - Want to know which value was chosen

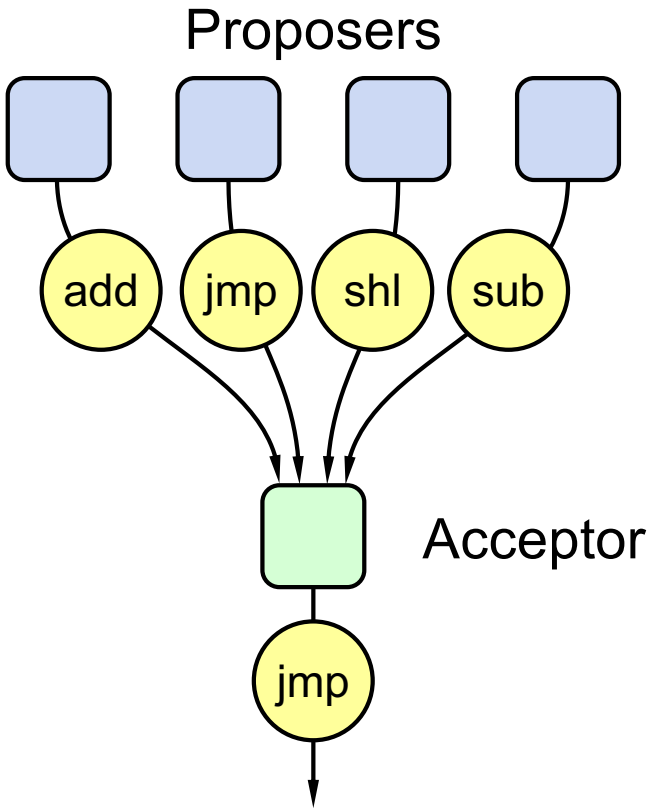
Clients talk to?

For this presentation:

- Each Paxos server contains both components

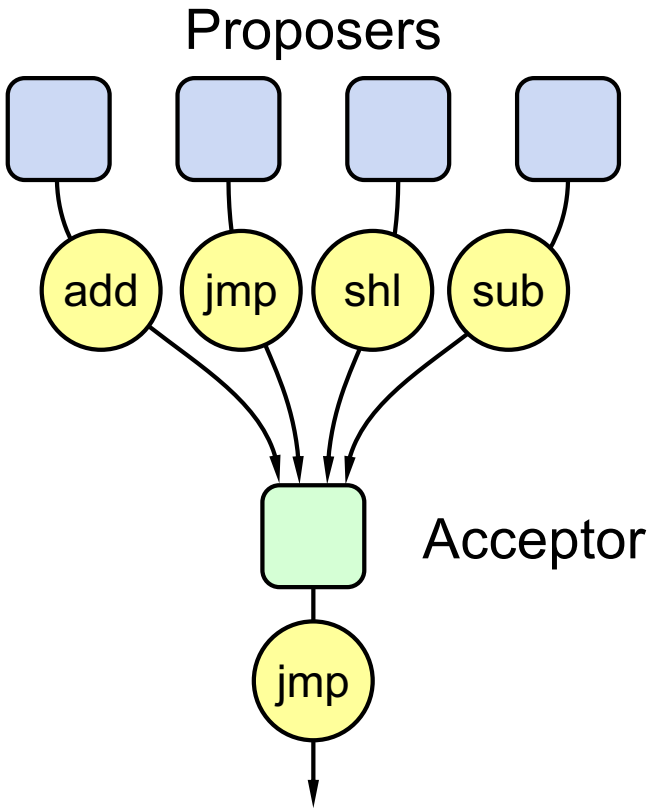
Build toward Correct Solution

- **Simple (incorrect) approach: a single acceptor chooses value**
- **Problems?**



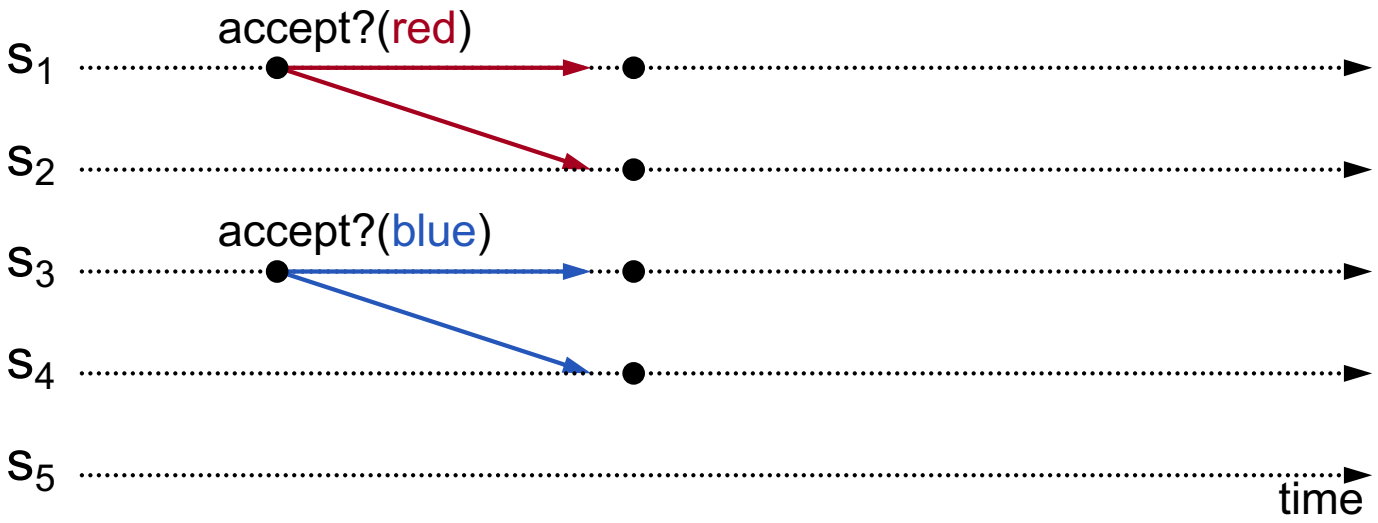
Build toward Correct Solution

- **Simple (incorrect) approach: a single acceptor chooses value**
- **What if acceptor crashes after choosing?**
- **Solution: quorum**
  - Multiple acceptors (3, 5, ...)
  - Value v is chosen if accepted by majority of acceptors
  - If one acceptor crashes, chosen value still available



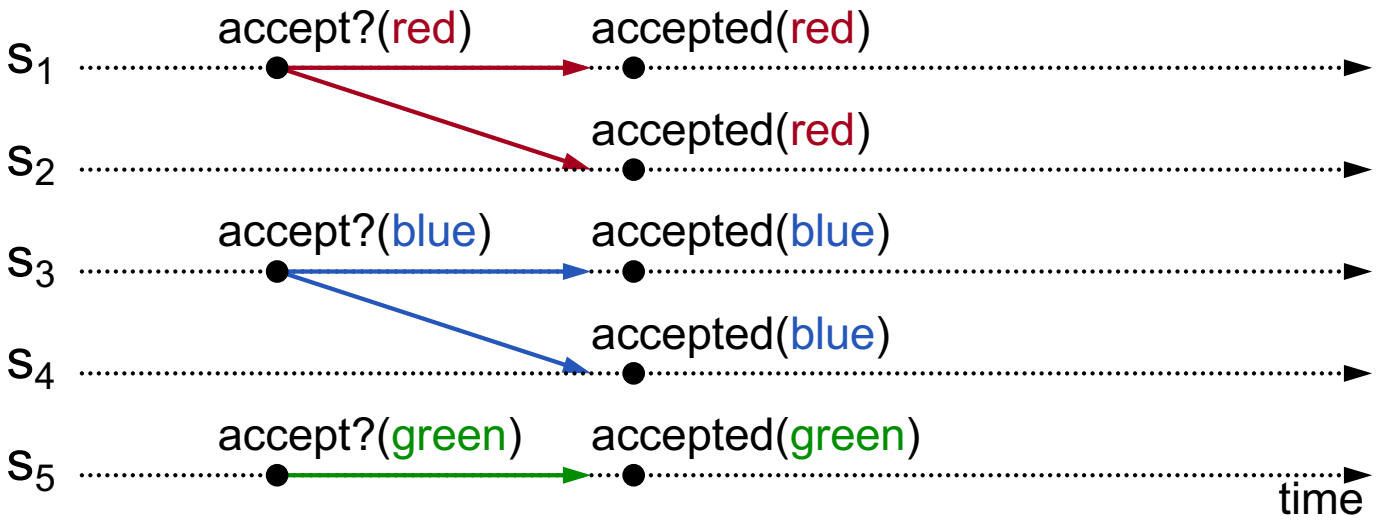
Approach 2: Accept First Proposal

- **Acceptor accepts only first value it receives?**
- **Problem?**



Approach 2: Accept First Proposal

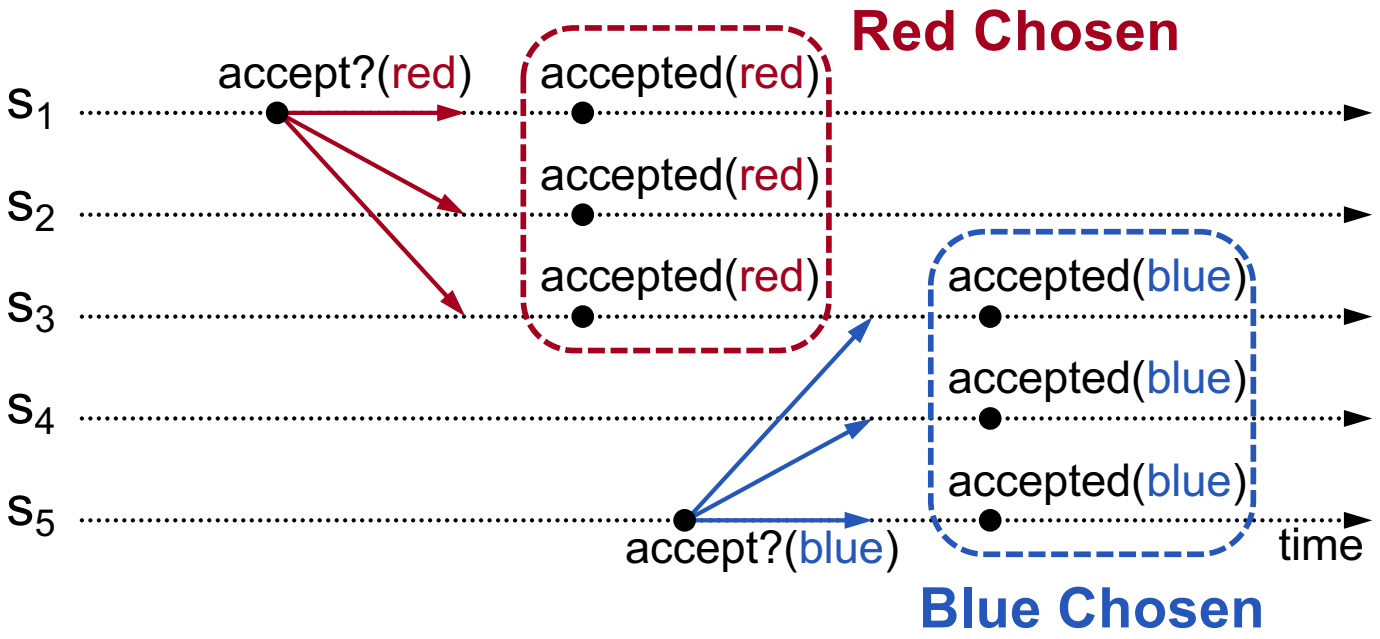
- **Acceptor accepts only first value it receives?**
- **If simultaneous proposals, no value might have majority**



Implication?
Acceptors must sometimes change mind

Approach 3: Accept Every Value?

- Acceptor accepts **every** value it receives?
- Problem? Could accept multiple values

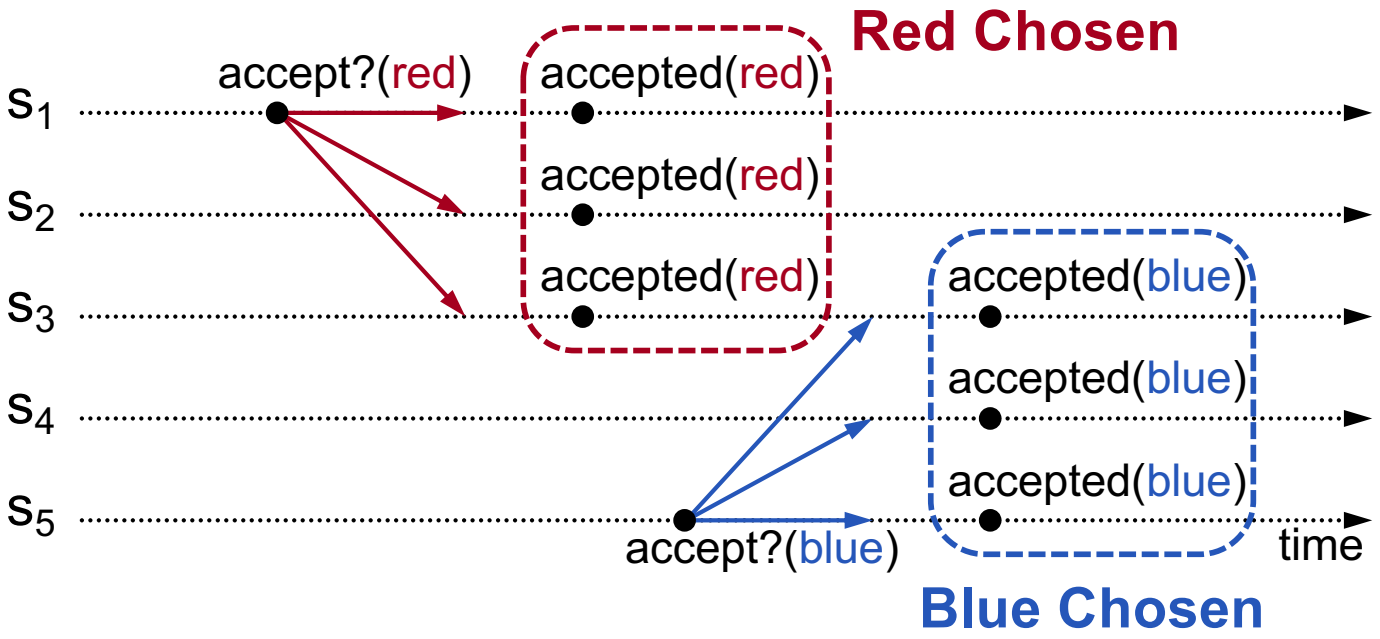


Implication?

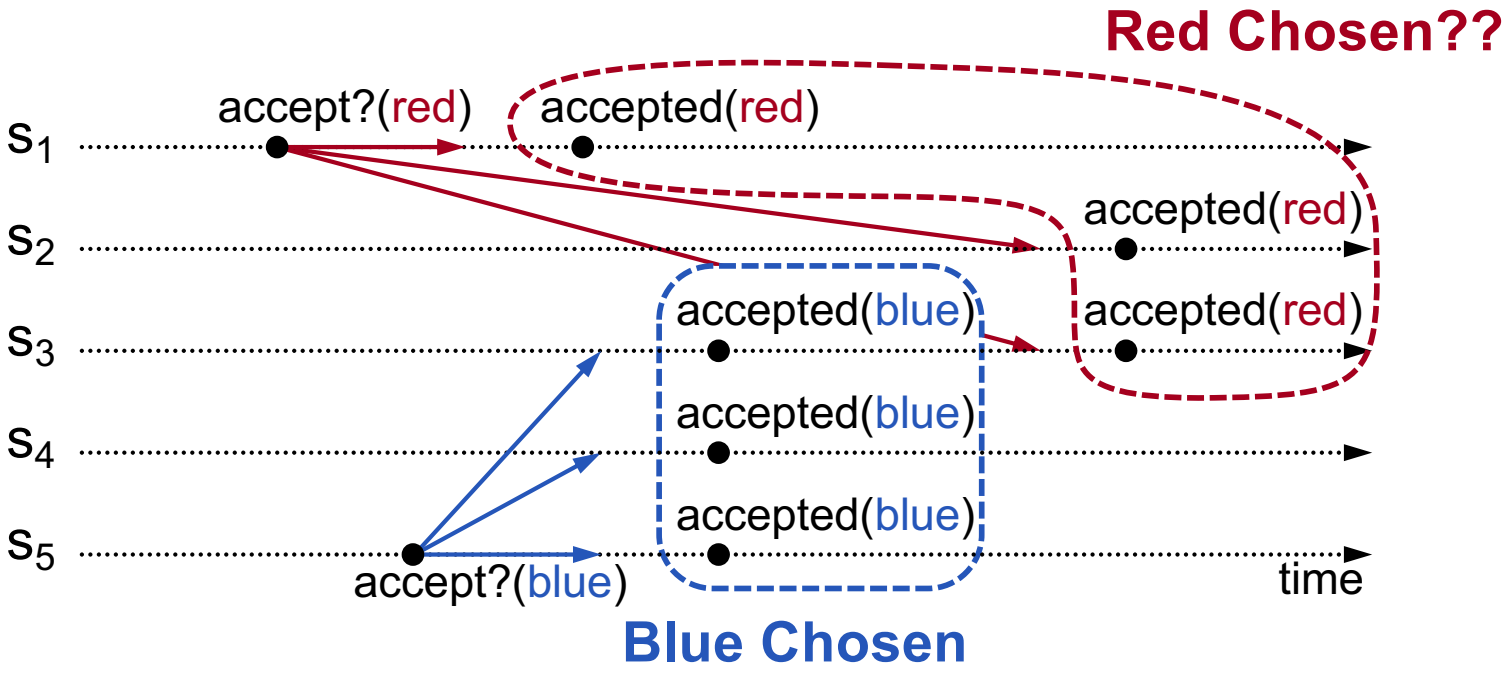
Once a value has been chosen, future proposals must propose that same value (2-phase protocol)

Approach 4: Discover?

- Once a value has been chosen, future proposals must propose that same value (2-phase protocol)



Approach 4: Discover?

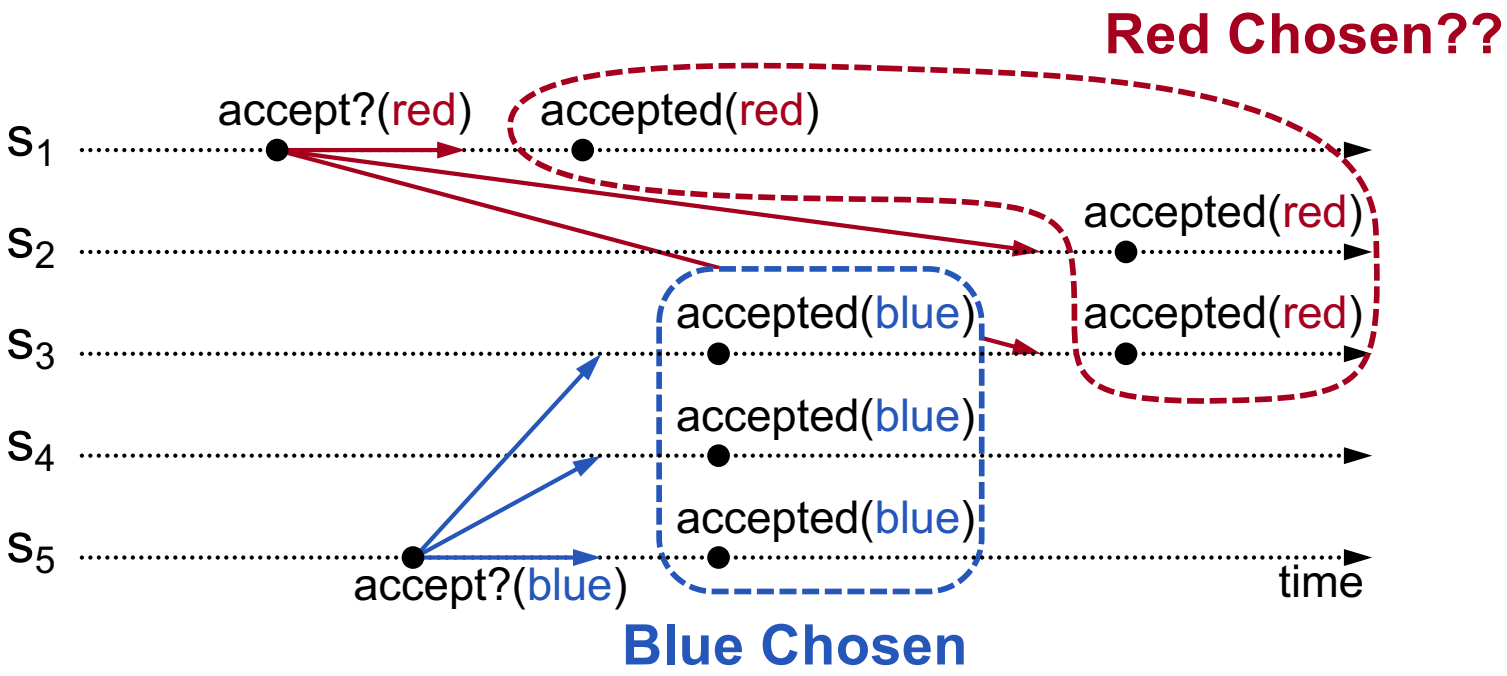


- s_5 needn't propose red (it hasn't been chosen yet)
- s_1's proposal must be aborted (s_3 must reject it)

Implication?

Must order proposals, reject old ones

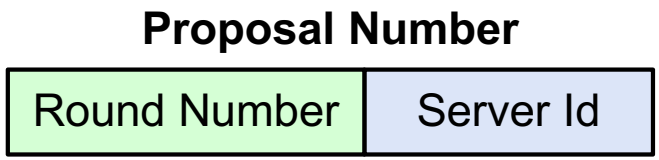
Basic Paxos



Must order proposals, reject old ones

Proposal Numbers

- **Each proposal has a unique number**
  - Higher numbers take priority over lower numbers
  - Higher numbers → Later proposal
  - It must be possible for a proposer to choose a new proposal number higher than anything it has seen/used before
- **One simple approach:**



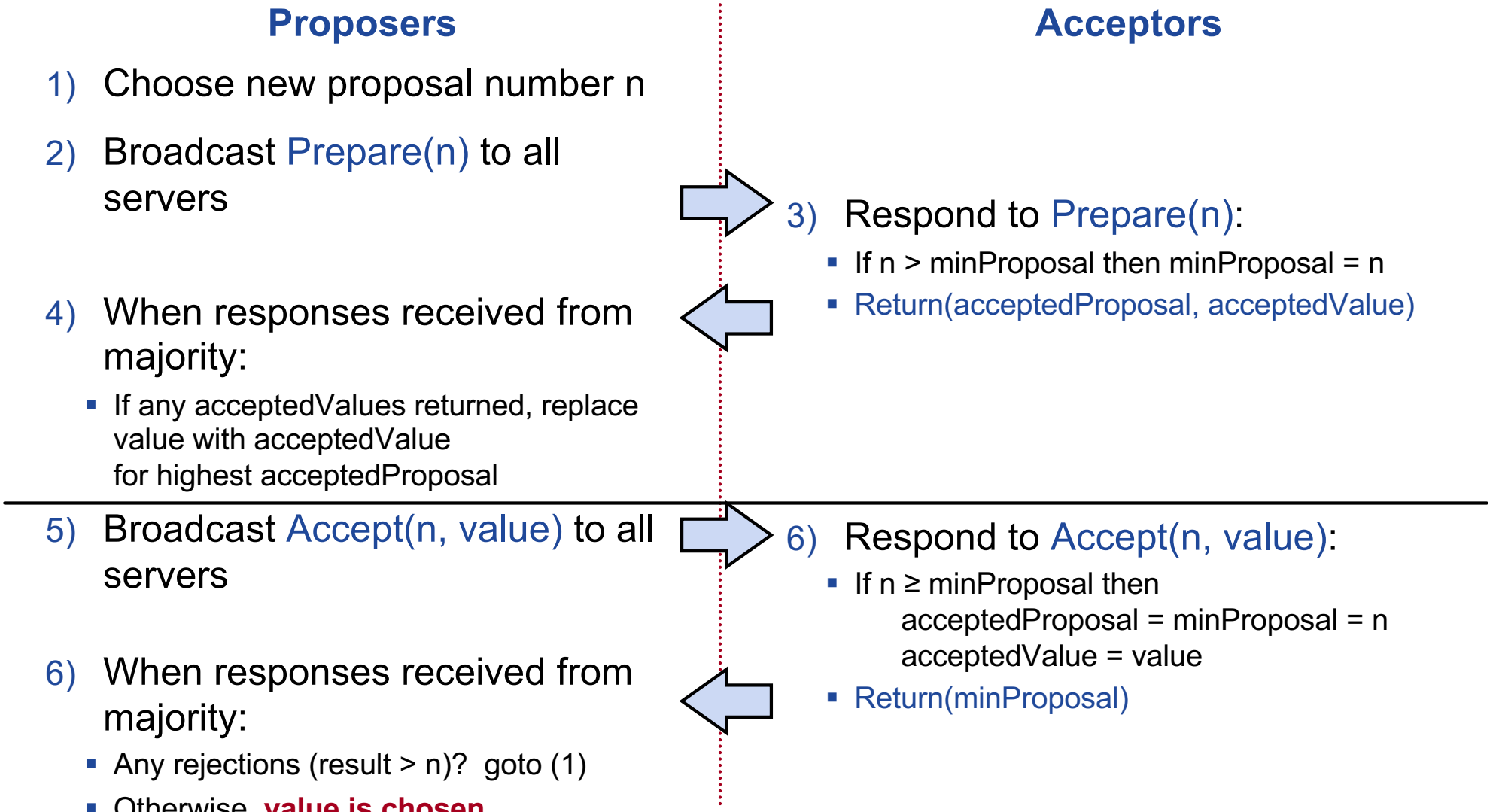
  - Each server stores maxRound: the largest Round Number it has seen so far
  - To generate a new proposal number:
    - Increment maxRound
    - Concatenate with Server Id
  - Proposers must persist maxRound on disk: must not reuse proposal numbers after crash/restart

Basic Paxos

Two-phase approach:

- **Phase 1: broadcast Prepare RPCs**
  - Find out about any (potentially) chosen values
  - Block older proposals that have not yet completed
- **Phase 2: broadcast Accept RPCs**
  - Ask acceptors to accept a specific value

Basic Paxos

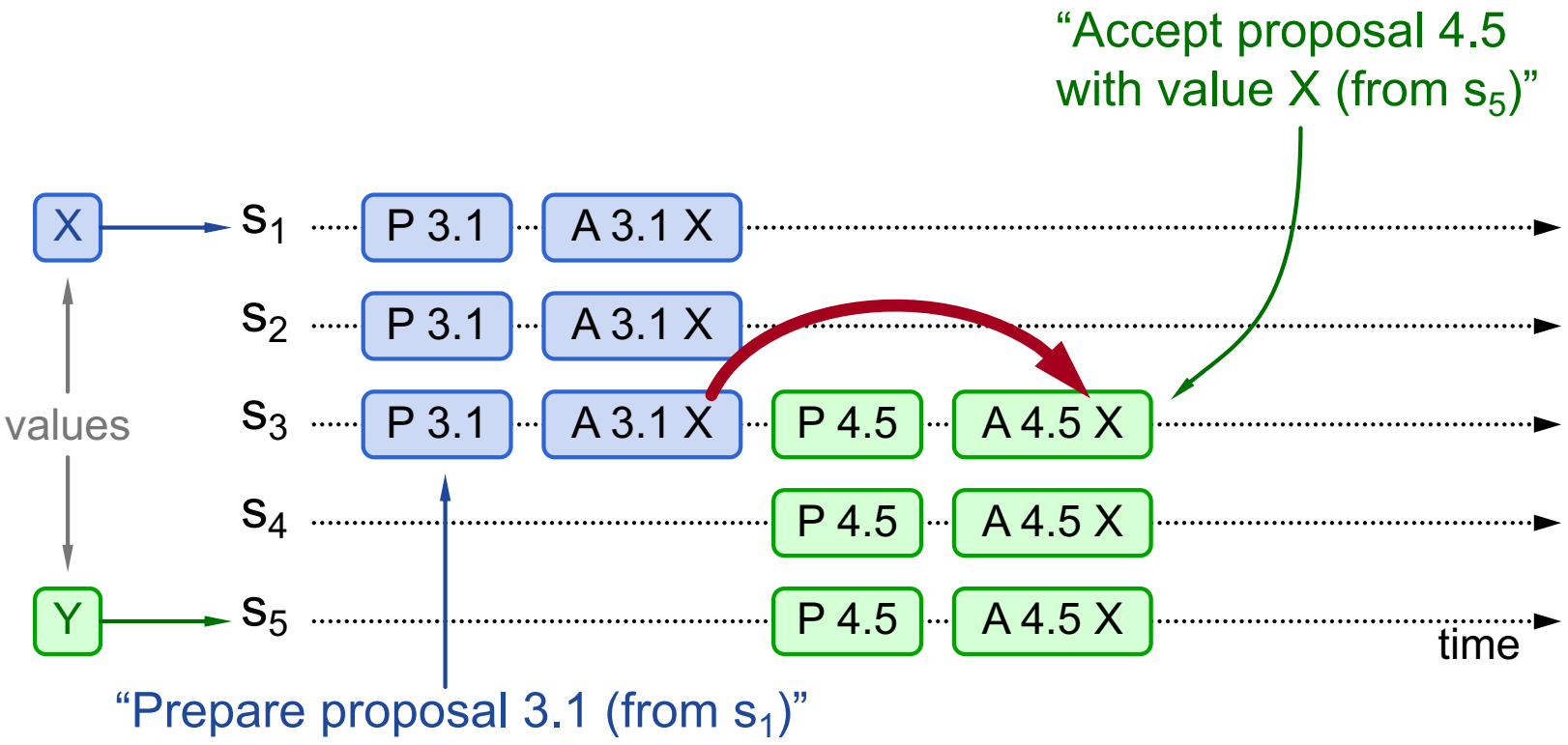


Acceptors must record minProposal, acceptedProposal, and acceptedValue on stable storage (disk)

Basic Paxos Example 1

Three possibilities when later proposal prepares:

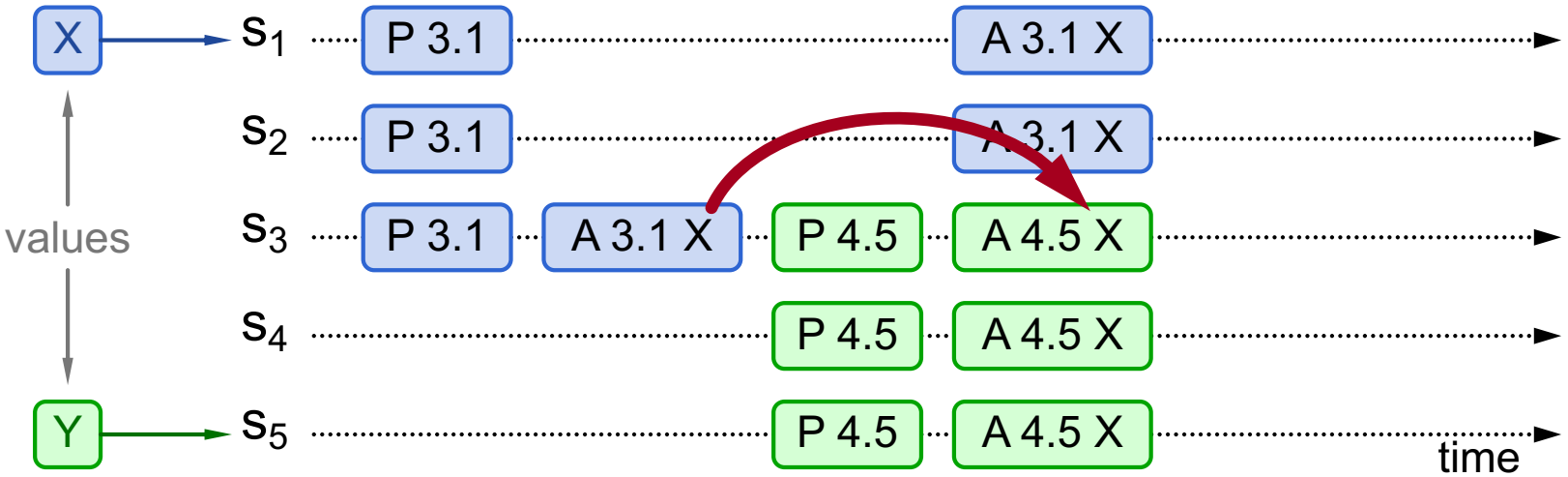
1. **Previous value already chosen:**
   - New proposer will find it and use it



Basic Paxos Example 2

Three possibilities when later proposal prepares:

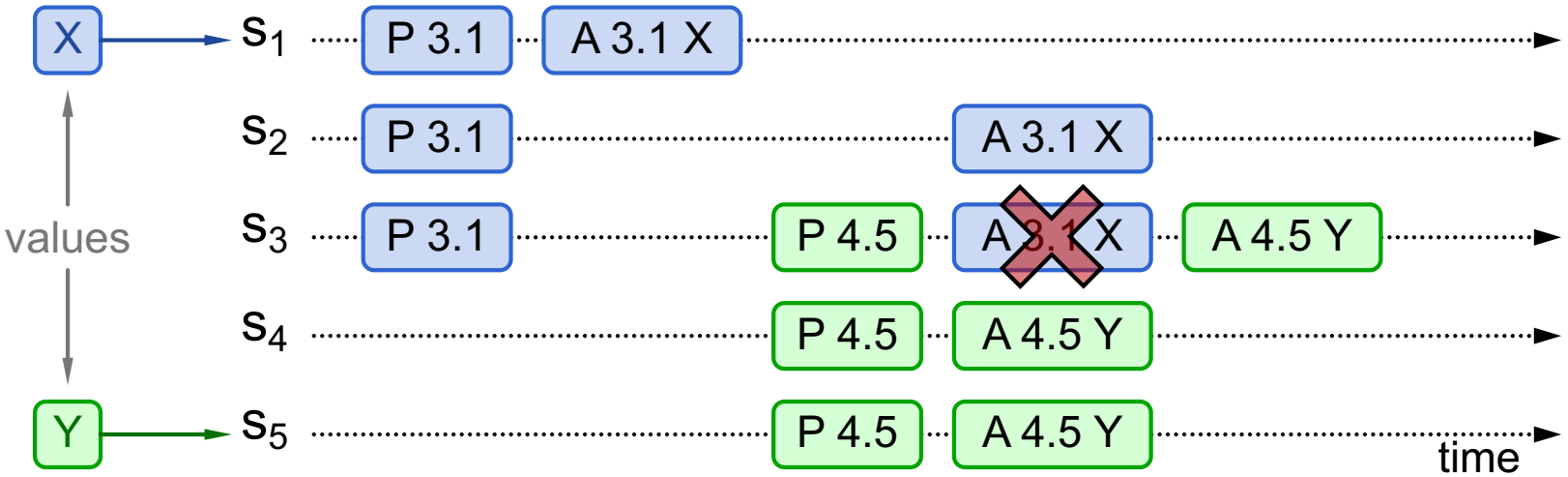
2. **Previous value not chosen, but later proposer sees it:**
   - New proposer will use existing value
   - Both proposers can succeed



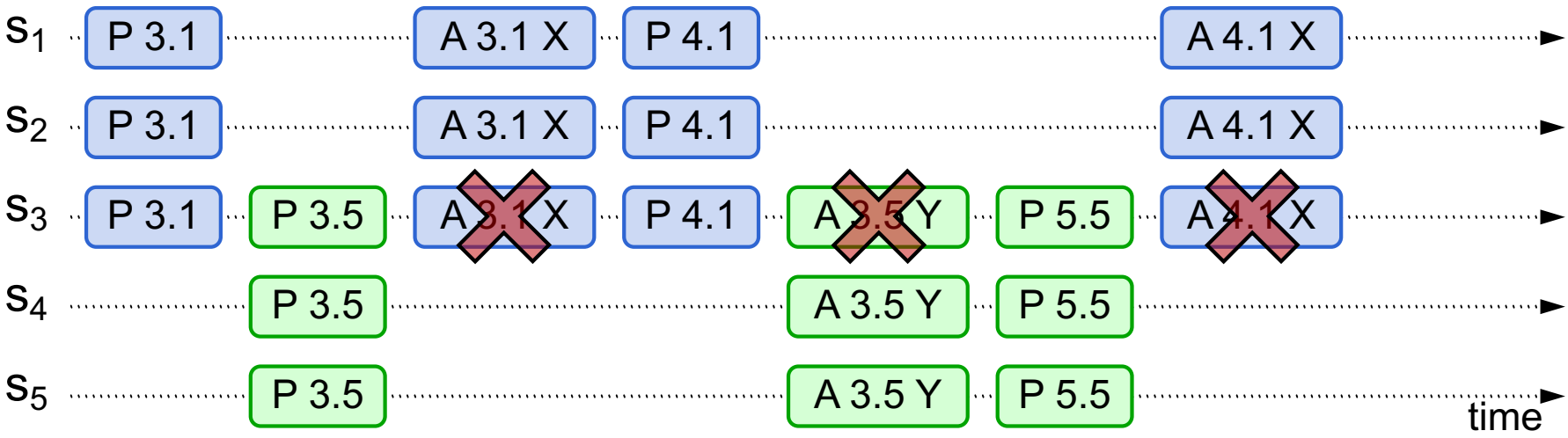
Basic Paxos Example 3

Three possibilities when later proposal prepares:

3. **Previous value not chosen, later proposer doesn't see it:**
   - New proposer chooses its own value
   - Older proposal blocked



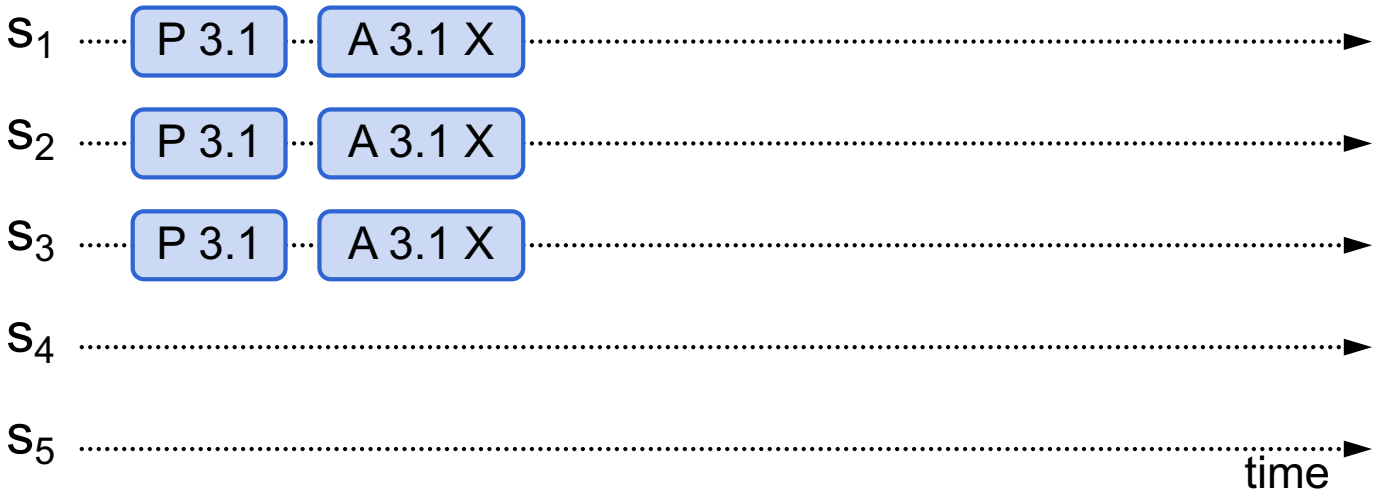
Liveness: Guaranteed?



- **One solution: randomized delay before restarting**
  - Give other proposers a chance to finish choosing
- **Multi-Paxos will use leader election instead**

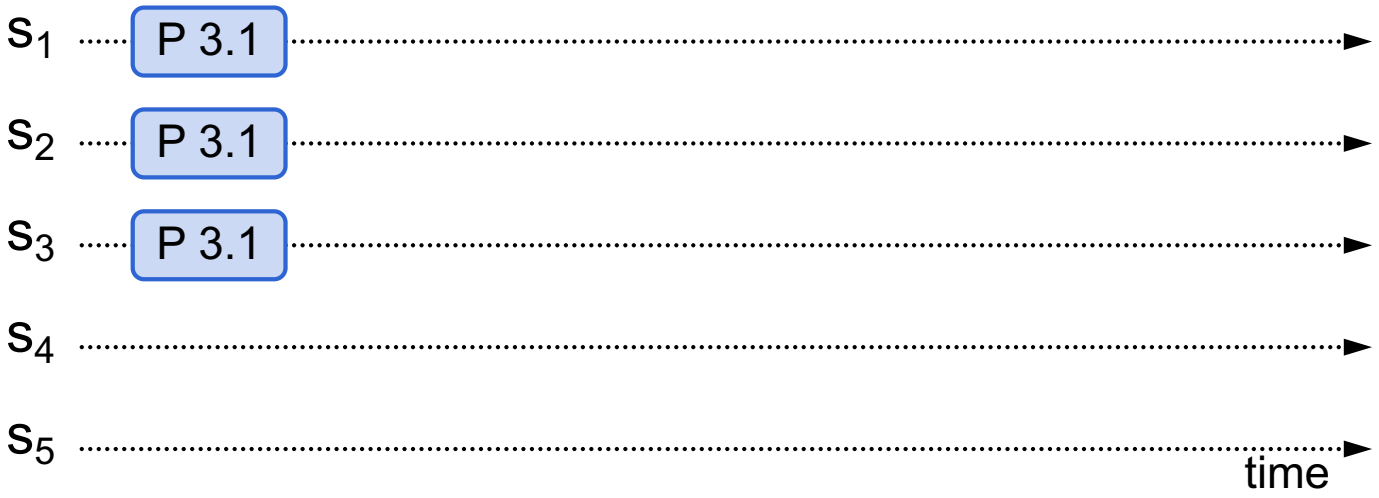
Question 1

- **After 3.1 X has been accepted, can a different server accept a different value of Y?**



Question 2

- **After 3.1 X has been proposed, what if s1 crashes and proposes 3.1 Y; is this safe?**



Question 3: Safe?

Acceptors

3) Respond to Prepare(n):
   - If n > minProposal then minProposal = n
   - Return(acceptedProposal, acceptedValue)

6) Respond to Accept(n, value):
   - If n ≥ minProposal then
     acceptedProposal = minProposal = n
     acceptedValue = value
   - Return(minProposal)

Acceptors

3) Respond to Prepare(n):
   - If n > minProposal then minProposal = n
   - Return(acceptedProposal, acceptedValue)

6) Respond to Accept(n, value):
   - If n ≥ minProposal then
     acceptedProposal = n
     acceptedValue = value
   - Return(minProposal)

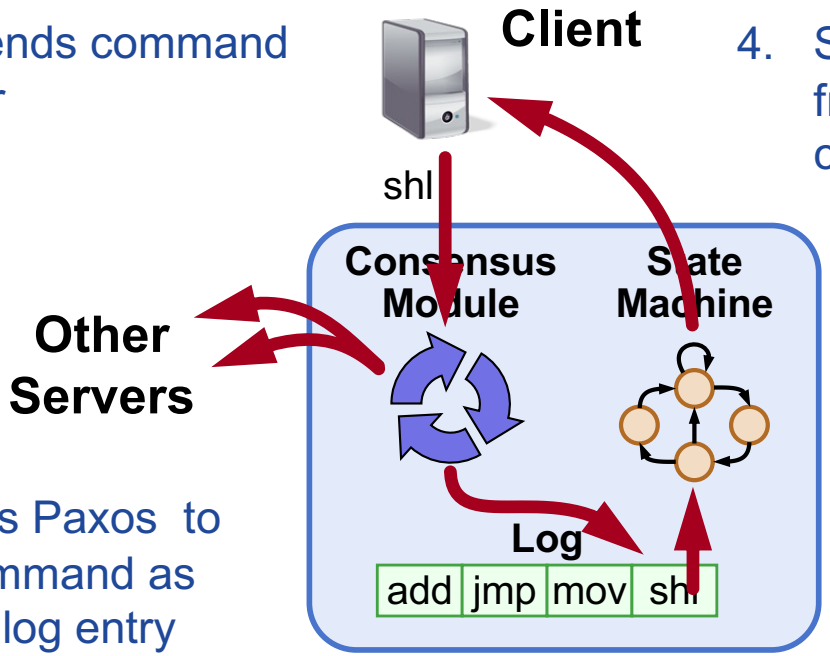
Other Notes

- Only proposer knows which value has been chosen
- If other servers want to know, must execute Paxos with their own proposal

Multi-Paxos

- **Separate instance of Basic Paxos for each entry in the log:**
  - Add index argument to Prepare and Accept (selects entry in log)

1. Client sends command to server



4. Server returns result from state machine to client

2. Server uses Paxos to choose command as value for a log entry

3. Server waits for previous log entries to be applied, then applies new command to state machine

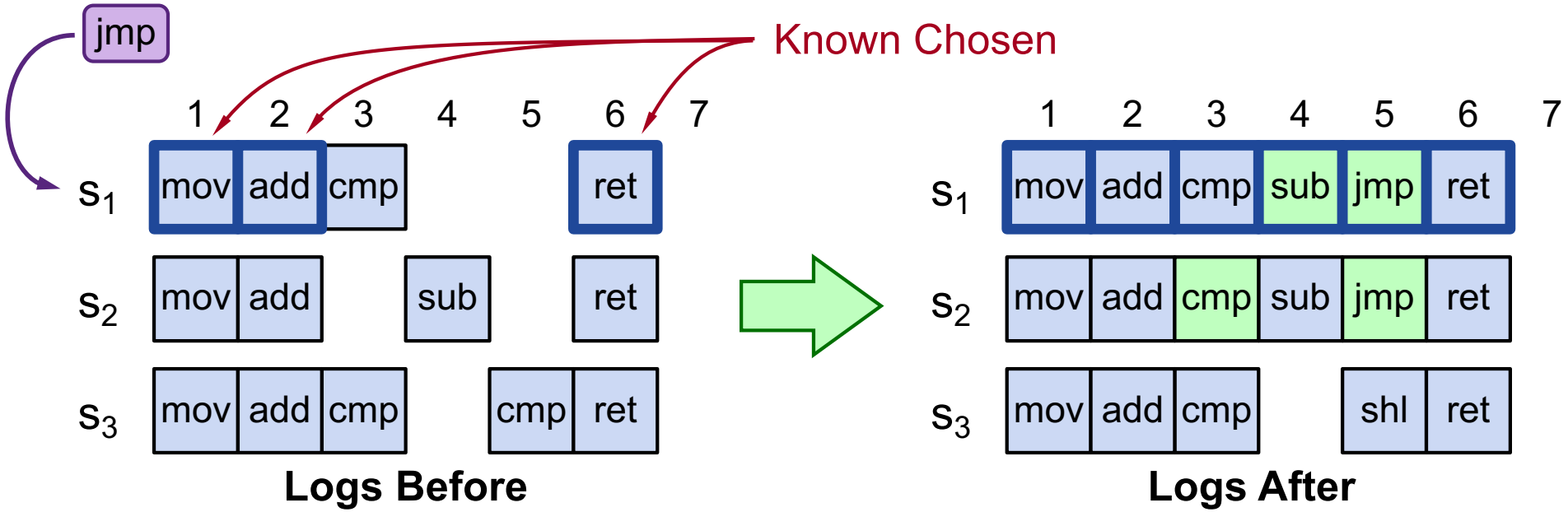
Multi-Paxos Issues

- **Which log entry to use for a given client request?**
- **Performance optimizations:**
  - Use leader to reduce proposer conflicts
  - Eliminate most Prepare requests
- **Ensuring full replication**
- **Client protocol**
- **Configuration changes**

Note: Multi-Paxos not specified precisely in literature

Selecting Log Entries

- **When request arrives from client:**
  - Find first log entry not known to be chosen
  - Run Basic Paxos to propose client's command for this index
  - Prepare returns acceptedValue?
    - Yes: finish choosing acceptedValue, start again
    - No: choose client's command



Selecting Log Entries

- Concurrent requests from clients?
- Apply requests to state machine concurrently?

Selecting Log Entries

- **Servers can handle multiple client requests concurrently:**
  - Select different log entries for each
- **Must apply commands to state machine in log order**

Improving Efficiency

- **Using Basic Paxos is inefficient:**
  - With multiple concurrent proposers, conflicts and restarts are likely (higher load → more conflicts)
  - 2 rounds of RPCs for each value chosen (Prepare, Accept)

Solution:

1. **Pick a leader**
   - At any given time, only one server acts as Proposer
2. **Eliminate most Prepare RPCs**
   - Prepare once for the entire log (not once per entry)
   - Most log entries can be chosen in a single round of RPCs

Leader Election

One simple approach from Lamport:

- **Let the server with highest ID act as leader**
- **Each server sends a heartbeat message to every other server every T ms**
- **If a server hasn't received heartbeat from server with higher ID in last 2T ms, it acts as leader:**
  - Accepts requests from clients
  - Acts as proposer and acceptor
- **If server not leader:**
  - Rejects client requests (redirect to leader)
  - Acts only as acceptor

Eliminating Prepares

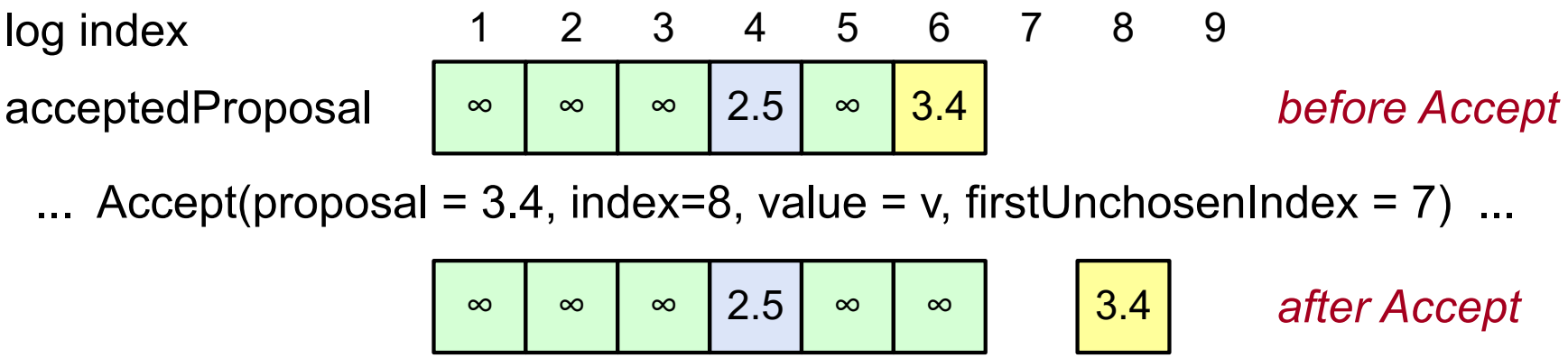
- **Why is Prepare needed?**
  - Block old proposals
    - Make proposal numbers refer to the entire log, not just one entry
  - Find out about (possibly) chosen values
    - Return highest proposal accepted for current entry
    - Also return noMoreAccepted: no proposals accepted for any log entry beyond current one
- **If acceptor responds to Prepare with noMoreAccepted, skip future Prepares with that acceptor (until Accept rejected)**
- **Once leader receives noMoreAccepted from majority of acceptors, no need for Prepare RPCs**
  - Only 1 round of RPCs needed per log entry (Accepts)

Full Disclosure

- **So far, information flow is incomplete:**
  - Log entries not fully replicated (majority only)
    Goal: full replication
  - Only proposer knows when entry is chosen
    Goal: all servers know about chosen entries
- **Solution part 1/4: keep retrying Accept RPCs until all acceptors respond (in background)**
  - Fully replicates most entries
- **Solution part 2/4: track chosen entries**
  - Mark entries that are known to be chosen:
    acceptedProposal[i] = ∞
  - Each server maintains firstUnchosenIndex: index of earliest log entry not marked as chosen

Full Disclosure, cont'd

- **Solution part 3/4: proposer tells acceptors about chosen entries**
  - Proposer includes its firstUnchosenIndex in Accept RPCs.
  - Acceptor marks all entries i chosen if:
    - i < request.firstUnchosenIndex
    - acceptedProposal[i] == request.proposal
  - Result: acceptors know about *most* chosen entries



Still don't have complete information

Full Disclosure, cont'd

- **Solution part 4/4: entries from old leaders**
  - Acceptor returns its firstUnchosenIndex in Accept replies
  - If proposer's firstUnchosenIndex > firstUnchosenIndex from response, then proposer sends Success RPC (in background)
- **Success(index, v): notifies acceptor of chosen entry:**
  - acceptedValue[index] = v
  - acceptedProposal[index] = ∞
  - return firstUnchosenIndex
  - Proposer sends additional Success RPCs, if needed

Client Protocol

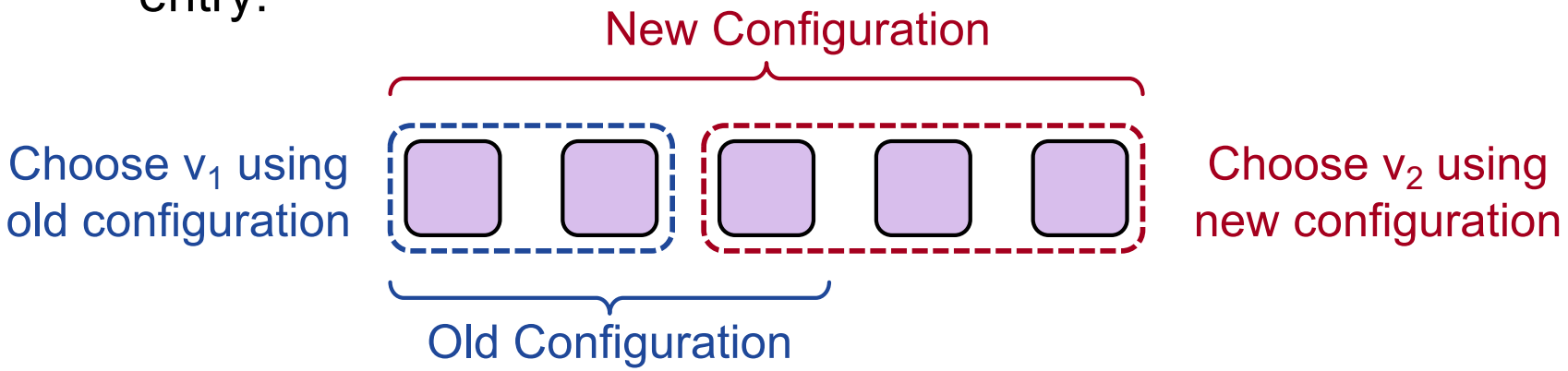
- **Send commands to leader**
  - If leader unknown, contact any server
  - If contacted server not leader, it will redirect to leader
- **Leader does not respond until command has been chosen for log entry and executed by leader's state machine**
- **If request times out (e.g., leader crash):**
  - Client reissues command to some other server
  - Eventually redirected to new leader
  - Retry request with new leader

Client Protocol, cont'd

- **What if leader crashes after executing command but before responding?**
  - Must not execute command twice
- **Solution: client embeds a unique id in each command**
  - Server includes id in log entry
  - State machine records most recent command executed for each client
  - Before executing command, state machine checks to see if command already executed, if so:
    - Ignore new command
    - Return response from old command
- **Result: exactly-once semantics as long as client doesn't crash**

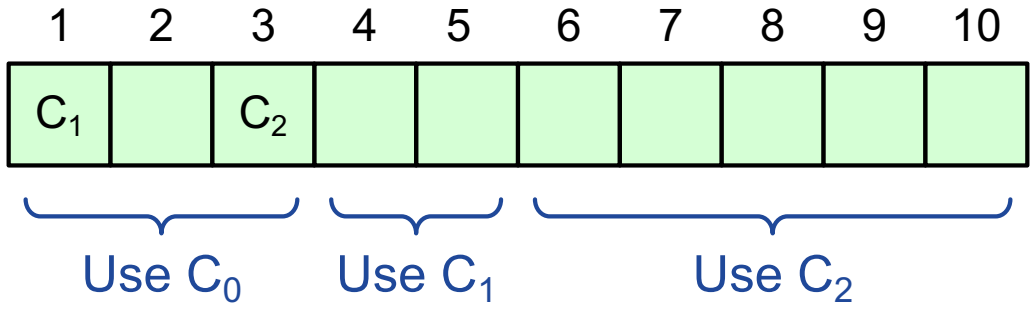
Configuration Changes

- **System configuration:**
  - ID, address for each server
  - Determines what constitutes a majority
- **Consensus mechanism must support changes in the configuration:**
  - Replace failed machine; Change degree of replication
- **Safety requirement:**
  - During configuration changes, it must not be possible for different majorities to choose different values for the same log entry:



Configuration Changes, cont'd

- **Paxos solution: use the log to manage configuration changes:**
  - Configuration is stored as a log entry
  - Replicated just like any other log entry
  - Configuration for choosing entry i determined by entry i-α. Suppose α = 3:



Paxos Summary

- **Basic Paxos:**
  - Prepare phase
  - Accept phase
- **Multi-Paxos:**
  - Choosing log entries
  - Leader election
  - Eliminating most Prepare requests
  - Full information propagation
- **Client protocol**
- **Configuration changes**